

ADR 001: Cómputo basado en Auto Scaling Group (EC2) vs. Contenedores (ECS/Fargate)

- **Fecha:** 2026-08-17
- **Estado:** Aceptado
- **Contexto:** Se requiere migrar una aplicación web monolítica tradicional desde un entorno On-Premises hacia AWS asegurando alta disponibilidad, capacidad de autorecuperación y simplicidad operativa en la fase inicial.

Decisión

Desplegamos el servicio web en instancias EC2 bajo un Auto Scaling Group (ASG) gestionado por Launch Templates.

Alternativas Evaluadas

- **AWS ECS (Elastic Container Service) con Fargate:** Requería dockerizar la aplicación, configurar un pipeline de CI/CD para build/push de imágenes a ECR y rediseñar la gestión de configuración del servicio web.
- **Instancias EC2 Estáticas:** Menor complejidad inicial, pero sin tolerancia a fallos ni capacidad de autorrecuperación horizontal.

Trade-off

- **Qué resignamos:** Mayor tiempo de arranque en eventos de escalado (*bootstrapping* de SO vs. segundos de un contenedor) y mayor sobrecarga en parches a nivel de sistema operativo. A cambio, evitamos la reingeniería a contenedores y mantenemos paridad directa con la operación On-Premises.

ADR 002: Persistencia Compartida con Amazon EFS vs. Almacenamiento Local (EBS)

- **Fecha:** 2026-08-17
- **Estado:** Aceptado
- **Contexto:** Múltiples instancias creadas dinámicamente por el ASG deben servir concurrentemente el mismo contenido web (`/var/www/html`) sin discrepancias ni necesidad de sincronizaciones manuales.

Decisión

Utilizamos Amazon Elastic File System (EFS) montado mediante NFS en todas las instancias del grupo de escalado.

Alternativas Evaluadas

- **Volúmenes EBS por Instancia:** Cada nodo almacena su copia local; requiere scripts de sincronización periódicos (`rsync/cron`) propensos a condiciones de carrera o inconsistencias de caché.

Trade-off

- **Qué resignamos:** Menor velocidad de operaciones de lectura/escritura (latencia típica de NFS vs. SSD local NVMe) y un costo por GB superior a EBS. A cambio, logramos consistencia de datos inmediata (*Read-Write-Many*) entre todas las zonas de disponibilidad.

ADR 003: Almacenamiento de Scripts de Bootstrap en Amazon S3 vs. AMIs Personalizadas (Baking)

- **Fecha:** 2026-08-17
- **Estado:** Aceptado
- **Contexto:** Se necesita aprovisionar, configurar e iniciar Apache en las instancias de forma automatizada al momento de su lanzamiento.

Decisión

Almacenamos los scripts de inicialización (`setup_web.sh`) en un Bucket de Amazon S3 desacoplado y los consumimos vía UserData mediante un IAM Instance Profile.

Alternativas Evaluadas

- **Golden AMIs (Packer/EC2 Image Builder):** Pre-empaquetar todo el software en la imagen base. Hace que el arranque sea rápido, pero cualquier ajuste menor en la configuración obliga a generar y versionar una nueva AMI.
- **UserData inline en Terraform:** Incrustar todo el script directamente en el Launch Template, dificultando pruebas locales y versionado modular de scripts.

Trade-off

- **Qué resignamos:** Dependencia de conectividad de red y salida a Internet/S3 durante el arranque de la instancia. A cambio, obtenemos inmutabilidad del código de aprovisionamiento y actualizaciones inmediatas sin regenerar imágenes de AMI.

ADR 004: Balanceador de Carga Application Load Balancer (ALB) vs. Network Load Balancer (NLB)

- **Fecha:** 2026-08-17
- **Estado:** Aceptado
- **Contexto:** Se requiere distribuir tráfico HTTP/HTTPS entre las instancias del ASG evaluando la disponibilidad real de la aplicación web.

Decisión

Implementamos un Application Load Balancer (ALB) en la capa pública como punto de entrada de la arquitectura.

Alternativas Evaluadas

- **Network Load Balancer (NLB - Capa 4):** Máximo rendimiento y latencia ultrabaja a nivel TCP, pero sin capacidad de inspección de cabeceras HTTP, enrutamiento por rutas/hosts ni chequeos de salud a nivel de código de estado HTTP (200 OK).
- **Classic Load Balancer (CLB):** Servicio heredado con menor flexibilidad de reglas y sin soporte moderno para Target Groups avanzados.

Trade-off

- **Qué resignamos:** No disponemos de IPs elásticas estáticas directas por zona como ofrece NLB y sumamos una latencia marginalmente mayor (Capa 7). A cambio, ganamos *Health Checks* a nivel de URL (/), descarga de SSL/TLS (ACM) y encadenamiento granular de Security Groups hacia las EC2.